WarpFix

AUTONOMOUS CI REPAIR

Your CI pipeline fixes itself.

WarpFix detects failed workflows, generates safe patches, validates them in an isolated sandbox, and opens a pull request — automatically.

GitHub App | 21 engines | sandbox-validated patches | fingerprint learning

Hardy Lebsack · Founder

warpfix.org

THE PROBLEM

CI breaks. Engineers become log readers.

- **Every red run is a context switch.** Read the logs, reproduce locally, guess a fix, push, wait for CI again.
- **Review tools comment; they don’t repair.** CodeRabbit, Snyk and friends tell you what is wrong — the pipeline stays broken.
- **The same failures repeat across repos.** Flaky tests, dependency breaks and type errors are re-diagnosed from scratch every time.

> CI repair is the missing half of the developer toolchain: review tools surface problems — nothing closes the loop back to a green pipeline.

THE PRODUCT

From failed workflow to merged fix, without a human in the loop.

01
Detect
GitHub webhook fires the moment a workflow fails.

→

02
Classify
Logs parsed, error fingerprinted, known fix looked up.

→

03
Patch
LLM writes a minimal, scoped change.

→

04
Validate
Clean sandbox: install deps, run the suite, score confidence.

→

05
Ship
Pull request opened with evidence attached.

Fingerprint learning

Each error is normalized and hashed. When the pattern returns, the proven fix is reused instantly — no LLM call, no wait.

Deterministic first

Lint, format and import fixes run through ESLint, Prettier and Ruff at zero LLM cost. AI is reserved for non-trivial repairs.

WHERE WE ARE

Live, bootstrapped, and already repairing.

- **Shipped product:** GitHub App, dashboard, and CLI (npx warpfix) — not a prototype.
- **Real coverage:** Node, Python, Go, Rust, JVM, Ruby, PHP, .NET, Swift and more across GitHub Actions, CircleCI, GitLab CI, Jenkins, Travis, Buildkite and Azure Pipelines.
- **Self-serve plans:** Free, Pro and Team tiers are live with subscription billing.

21	10+	0
SPECIALIZED ENGINES	CI PROVIDERS SUPPORTED	OUTSIDE FUNDING

WHY DAYTONA

Every patch has to earn its PR in a clean sandbox.

- **One sandbox per repair.** Fresh container, install the project's dependencies, run the failing suite, compare results, tear it down — 30 to 180 seconds, bursty and parallel.
- **Today this is our bottleneck.** Validation runs on a single 512 MB worker; JVM and large Maven/Gradle builds OOM mid-run, so verified repairs for those ecosystems are switched off.
- **What we need:** fast cold starts, per-second billing, snapshot restore for warmed dependency caches, and the ability to burst to dozens of concurrent short-lived sandboxes.

> Daytona's sub-90 ms sandbox creation and snapshot model fit this workload exactly — validate every patch, on every stack, without pre-warming a fleet.

THE ASK

Let us validate every patch, on every stack.

$10,000 in Daytona credits

- Run isolated patch validation across every supported ecosystem — including the JVM builds we currently can’t verify.
- Snapshot dependency caches per stack so validation starts warm, not from zero.
- Stress-test concurrency: dozens of parallel repairs during peak CI hours.

What you get back

Usage data and benchmarks, a written case study, and developer-facing content about running CI repair at scale on Daytona.

Where to look

warpfix.org · github.com/bytepassperks/warpfix-intelligence-cloud · support@warpfix.org